Universidad Nacional Autónoma de México

Facultad de Ciencias, 2027-I

Compiladores



Práctica 06

Equipo:

Delgado Díaz Hermes Alberto - 319258613
Medina Peralta Joaquín - 320202513

1 de Noviembre de 2026

1. Introducción y objetivos

En la Práctica 5 se incorporó al compilador de MiniC una infraestructura de análisis semántico para registrar declaraciones, administrar ámbitos y resolver identificadores. Sin embargo, un programa podía tener todas sus variables declaradas y seguir siendo incorrecto, por ejemplo al asignar un valor booleano a una variable entera.

El propósito de la Práctica 6 es ampliar esa infraestructura con **verificación estática de tipos**, sin volver a procesar caracteres ni tokens durante la etapa semántica. La comprobación se realiza sobre el árbol de sintaxis abstracta (AST) construido por el parser.

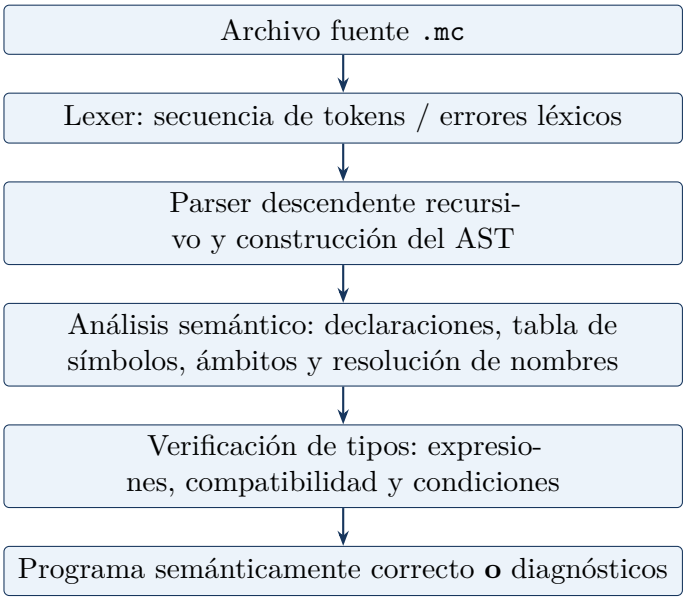
Objetivo general. Determinar el tipo de las expresiones y comprobar su compatibilidad con las reglas de MiniC en inicializaciones, asignaciones, operadores y condiciones.

Objetivos específicos.

- Reutilizar el lexer, el parser descendente recursivo, el AST y la tabla de símbolos.
- Representar los tipos de MiniC y calcular el tipo de cada expresión.
- Validar todas las categorías de operadores y las sentencias de control.
- Evitar diagnósticos en cascada mediante la propagación de `TYPE_ERROR`.
- Conservar el manejo de ámbitos, *shadowing*, diagnósticos y liberación de memoria.
- Comprobar el comportamiento con pruebas públicas y pruebas adicionales del equipo.

2. Arquitectura del compilador

La implementación está dividida en módulos. El programa principal de `src/main.c` coordina las etapas y detiene el análisis semántico si el lexer o el parser encontraron errores.



2.1. Distribución modular

Módulo	Responsabilidad real en el proyecto
`symbol.[ch]`	Construye y libera símbolos; almacena nombre, tipo declarado, ámbito, posición y referencia a declaración.
`scope.[ch]`	Crea y destruye ámbitos, con almacenamiento dinámico de símbolos y referencia al padre.
`symbol_table.[ch]`	Inserción, búsqueda local y visible, entrada/salida de ámbitos y propiedad de símbolos.
`semantic.[ch]`	Recorre sentencias, registra declaraciones, valida usos, coordina el verificador y emite errores.
`type.[ch]`	Define la enumeración de tipos, sus nombres y la compatibilidad exacta.
`type_checker.[ch]`	Calcula tipos de expresiones y valida operadores, inicializadores, asignaciones y condiciones.

Los archivos de cabecera se encuentran en `include/semantic/` y las implementaciones en `src/semantic/`. El proyecto conserva además `src/lexer/`, `src/parser/` y `src/ast/`.

3. Sistema de tipos y representación interna

MiniC es un lenguaje con tipado estático: los tipos se comprueban durante la compilación. En esta versión, sus valores pertenecen a las categorías `int` y `bool`. La implementación introduce además dos valores auxiliares, que no representan tipos declarables por el usuario:

```
typedef enum {
    TYPE_INT,
```

```
    TYPE_BOOL,
    TYPE_ERROR,
    TYPE_UNKNOWN
} Type;
```

Valor	Significado
`TYPE_INT`	Entero: literal entero, variable entera o resultado aritmético válido.
`TYPE_BOOL`	Booleano: literales lógicos, variables booleanas o comparación válida.
`TYPE_ERROR`	Resultado de una expresión con error; se propaga para no duplicar diagnósticos.
`TYPE_UNKNOWN`	Marcador auxiliar para situaciones no determinadas; no corresponde a una variable sin declaración.

La función `type_to_string` proporciona nombres legibles y consistentes en los mensajes de error. La compatibilidad se comprueba en `src/semantic/type.c`:

```
int types_are_compatible(Type expected, Type actual)
{
    return expected == actual &&
           expected != TYPE_ERROR &&
           expected != TYPE_UNKNOWN;
}
```

En consecuencia, *int* ↔ *int* y *bool* ↔ *bool* son compatibles, pero *int* ↔ *bool* no lo es. **No se realizan conversiones implícitas.**

4. Determinación del tipo de las expresiones

Se eligió la estrategia permitida por el enunciado de **devolver el tipo desde el recorrido**, en lugar de almacenarlo en cada nodo del AST. La función principal es:

```
Type type_check_expression(
    SemanticAnalyzer *analyzer,
    ASTNode *expression
);
```

El recorrido es recursivo y trabaja de las hojas a la raíz: un literal entero produce `TYPE_INT`, un literal booleano `TYPE_BOOL`, un identificador consulta la tabla de símbolos, y una expresión unaria o binaria combina los tipos de sus operandos según el operador.

Por ejemplo, para `5 + 2 * 3`, la multiplicación produce `int`, y luego la suma también produce `int`; para `5 < 2 + 3`, la suma produce `int` y el operador relacional produce `bool`. La precedencia y asociatividad ya están reflejadas en el AST que construye `parser_ast.c`.

4.1. Relación entre el AST y la tabla de símbolos

El AST conserva el nombre y la posición de cada uso de identificador. Al visitar ese nodo, `check_identifier` invoca `symbol_table_lookup`, que busca desde el ámbito actual hacia los padres y devuelve el símbolo visible más cercano. El tipo obtenido es `symbol->type`.

Cada `Symbol` también conserva una **referencia no propietaria** al nodo de declaración del AST. En esta implementación **no se añadió** un campo `inferred_type` a los nodos ni un puntero `resolved_symbol` a cada identificador: la resolución se hace al visitar el uso. Esto evita describir cambios al AST que el código realmente no contiene.

El AST sigue aportando tipo de nodo, operador, operandos y posiciones `line`/`column`. El *shadowing* se resuelve al buscar primero en el ámbito más interno; al abandonarlo vuelve a ser visible la declaración exterior.

5. Reglas de compatibilidad y operadores

5.1. Operadores

Categoría	Operadores	Operandos	Resultado
Aritméticos	`+`, `-`, `*`, `/`	`int`, `int`	`int`
Negativo unario	`-`	`int`	`int`
Relacionales	`<`, `<=`, `>`, `>=`	`int`, `int`	`bool`
Igualdad	`==`, `!=`	Dos tipos iguales	`bool`
Lógicos	`&&`, `||`	`bool`, `bool`	`bool`

Las reglas se implementaron en `check_binary` y `check_unary` de `type_checker.c`. Se analizan ambos operandos; si alguno es `TYPE_ERROR`, no se emite otro error de compatibilidad para el operador padre. Si ambos son válidos pero incompatibles, el diagnóstico muestra el operador y los tipos encontrados.

La igualdad admite `int == int` y `bool == bool`, pero no `int == bool`. En particular, la semántica de igualdad no convierte tipos.

5.2. Inicializadores y asignaciones

En una declaración `int x = expresion;` se registra primero el símbolo de `x`, se analiza el inicializador y se compara su tipo con el tipo declarado. Por ello, la implementación acepta desde el punto de vista de visibilidad `int x = x;`; el uso de un valor no inicializado no forma parte del alcance de la práctica.

Cuando se declara de nuevo un nombre en el mismo ámbito, se informa la redeclaración sin sustituir el símbolo original. Aun así, el inicializador se analiza para hallar errores independientes y se coteja con el tipo escrito en la nueva declaración.

En una asignación `x = expresion;` se busca primero el símbolo de destino y se analiza la expresión derecha aun si el destino no existe. Cuando el destino es inexistente, no se produce un segundo error de incompatibilidad basado en un tipo desconocido.

5.3. Condiciones y sentencia `print`

Las condiciones de `if` y `while` deben ser `bool`; un valor `int`, aun distinto de cero, es inválido. Las ramas de `if` y el cuerpo de `while` se revisan semánticamente sin ejecutar el programa, incluso si la condición contiene un error. La instrucción `print` admite expresiones `int` y `bool` semánticamente correctas.

6. Propagación de errores y recuperación

Para evitar errores derivados de uno anterior, cuando falla la resolución de un identificador o la validación de una operación, el tipo devuelto es `TYPE_ERROR`. Los operadores superiores reconocen ese valor y omiten nuevos mensajes basados en el mismo problema; sin embargo, **siguen recorriendo los subárboles independientes**.

Caso del equipo `tests/team/casos/t19_propagacion_sin_cascada.mc`:

```
print(no_existe + true);
int n = 1 + false;
```

```
Error semantico [1:6]: la variable 'no_existe' no ha
  sido declarada.
Error semantico [2:10]: el operador '+' requiere
  operandos de tipo 'int', pero se encontraron 'int'
   y 'bool'.
```

La primera expresión produce un error por el identificador inexistente, sin un mensaje adicional de suma incompatible. La segunda contiene un error independiente y se reporta.

La recuperación sintáctica heredada también permite continuar después de un inicializador incompleto. El caso `t20_recuperacion_sintactica.mc` contiene dos líneas inválidas separadas por una declaración válida:

```
int x = ;
bool activo = true;
int y = ;
print(activo);
```

```
Error sintactico [1:8]: se esperaba una expresion,
  pero se encontro ';'.
```

```
Error sintactico [3:8]: se esperaba una expresion,
  pero se encontro ';'.
```

La corrección en `parser_ast.c` evita construir una declaración incompleta cuando una expresión obligatoria devuelve `NULL`. Si existen errores léxicos o sintácticos, el AST parcial no se envía al análisis semántico.

7. Pruebas y resultados

Se separaron las pruebas originales del material de la práctica de las pruebas adicionales del equipo. El directorio `tests/public/` contiene los **24 casos públicos** (9 válidos y 15 inválidos); `tests/team/` contiene **20 casos propios** (8 válidos y 12 inválidos). Ambas carpetas incluyen manifest, entradas y ejecutores, y la del equipo incluye salidas esperadas específicas.

7.1. Cobertura de la batería del equipo

Casos	Comprobaciones
`t01`–`t04`	Programa vacío, precedencia, asociatividad, negativos anidados y booleanos.
`t05`–`t08`	Ámbitos anidados, *shadowing* con tipos distintos, control y autorreferencia.
`t09`–`t11`	Uso antes de declaración, acceso fuera de ámbito y redeclaración.
`t12`–`t18`	Errores de inicialización, asignación, aritmética, relacionales, igualdad, lógica y condiciones.
`t19`–`t20`	Prevención de errores en cascada y recuperación de dos errores sintácticos.

7.2. Ejecución automatizada

Comandos ejecutados desde la raíz de `Practica06/`:

```
make clean && make
make test-public
make test-team
# Equivalente a ejecutar ambos conjuntos:
make test
```

Batería	Correctas	Fallidas	Estado
Públicas (profesor)	24	0	Aprobado
Equipo	20	0	Aprobado
Total	**44**	**0**	Aprobado

La compilación se verificó con `gcc -std=c11 -Wall -Wextra -Wpedantic` sin advertencias del compilador. En la ejecución local del proyecto adjunto, ambos conjuntos terminaron sin casos fallidos.

Alcance de la validación. El ejecutor público verifica principalmente código de salida y contratos de `stdout`/`stderr`; su propio resumen indica que los diagnósticos detallados requieren revisión humana. El ejecutor del equipo compara **exactamente** el contenido de la salida estándar o de error contra sus archivos esperados, incluidas posiciones y cantidad de diagnósticos. Ninguno de los conjuntos sustituye pruebas privadas del profesor.

8. Manejo y verificación de memoria

El AST es propietario de sus nodos hijos, listas y cadenas copiadas; su liberación se realiza recursivamente con `ast_destroy`. Cada símbolo almacena su propio nombre y una referencia *no propietaria* al nodo de declaración del AST.

La tabla de símbolos mantiene dos relaciones: `parent`, que permite buscar identificadores visibles, y `next_owned`, que encadena los ámbitos que deben liberarse. Al salir de un bloque deja de ser visible el ámbito local, pero se conserva hasta `symbol_table_destroy`. De ese modo se mantiene una política de propiedad explícita sin liberar por error los nodos del AST.

En las pruebas realizadas en Linux Mint y compartidas durante el desarrollo se obtuvieron los siguientes resultados con **Valgrind/Memcheck**:

Archivo	Alloc/free	Bytes al salir	Errores
`p22_completo_valido.mc`	120/120	0	0
`p20_propagacion.mc`	71/71	0	0
`t20_recuperacion_sintactica.mc`	40/40	0	0

Nota: La tercera ejecución se realizó sobre el mismo contenido durante el desarrollo, antes de renombrarlo como el caso del equipo `t20`.

Comandos reproducibles con la estructura final:

```
make memcheck FILE=tests/public/casos/
  p22_completo_valido.mc
make memcheck FILE=tests/public/casos/p20_propagacion
  .mc
make memcheck FILE=tests/team/casos/
  t20_recuperacion_sintactica.mc
```

Los resultados anteriores corresponden a **casos ejecutados**, no a una prueba exhaustiva de todas las rutas. En los archivos inválidos, que MiniC devuelva un código distinto de cero no equivale a un error de memoria.

9. Uso de herramientas de inteligencia artificial

10. Uso de Inteligencia Artificial

Durante el desarrollo de la práctica se utilizó **Claude** como herramienta de apoyo para identificar y corregir errores críticos heredados de implementaciones anteriores, principalmente relacionados con la construcción del AST y la recuperación ante errores sintácticos.

Asimismo, se utilizó para proponer casos de prueba más exigentes que los habituales, combinando múltiples errores, ámbitos anidados, incompatibilidades de tipos y situaciones límite que permitieran evaluar la robustez del compilador.

Las sugerencias fueron revisadas, implementadas y verificadas por el equipo mediante pruebas automatizadas, compilación con GCC y análisis de memoria con Valgrind.

11. Conclusiones

La Práctica 6 amplió el análisis semántico de MiniC desde la resolución de nombres hasta la comprobación estática de tipos. La separación entre `semantic.c` y `type_checker.c` permite coordinar el recorrido del AST y mantener las reglas de operadores en un módulo específico. El uso de `TYPE_ERROR` evita que un fallo inicial se convierta en múltiples diagnósticos redundantes sin ocultar problemas independientes.

El compilador acepta expresiones y asignaciones compatibles, rechaza operaciones que mezclan enteros y booleanos y comprueba condiciones de tipo `bool`. Los **44 casos de prueba** ejecutados terminaron correctamente según sus respectivos contratos, y las ejecuciones seleccionadas con Valgrind no mostraron fugas ni errores de memoria. Con esto se cumplen las responsabilidades previstas para el *front-end* de MiniC dentro del alcance documentado; no se implementaron ejecución, optimizaciones ni generación de código.

Referencias y materiales utilizados

1. Pablo Martínez, Y. J. y Salazar González, P. Y. *Compiladores, Práctica 6*. Facultad de Ciencias, UNAM, semestre 2027-1.
2. LLVM Project. *Kaleidoscope: Implementing a Parser and AST*. LLVM Documentation. Disponible en: https://llvm.org/docs/tutorial/MyFirstLanguageFrontend/LangImpl02.html
3. Nystrom, R. *Crafting Interpreters: Resolving and Binding*. Genever Benning, 2021. Disponible en: https://craftinginterpreters.com/resolving-and-binding.html
4. Cornell University. *Introduction to Compilers: Type Systems*. CS 4120, Spring 2026. Disponible en: https://www.cs.cornell.edu/courses/cs4120/2026sp/notes.html?id=typesystems
5. Cornell University. *Functional Programming in OCaml: Type Checking*. CS 3110, Spring 2021. Disponible en: https://courses.cs.cornell.edu/cs3110/2021sp/textbook/interp/typecheck.html
6. Valgrind Developers. *Valgrind User Manual: Memcheck, a Memory Error Detector*. Valgrind Documentation. Disponible en: https://valgrind.org/docs/manual/mc-manual.html